DESIGN AND ANALYSIS OF ALGORITHMS
UNIT – 1

Syllabus:
Algorithm, Pseudo code for expressing algorithms, performance Analysis-Space complexity, Time complexity, Asymptotic Notation- Big oh notation, Omega notation, Theta notation and Little oh notation, Amortized analysis.

Algorithm:

Definition-1: An algorithm is a sequence of unambiguous instructions for solving a problem,

i.e., for obtaining a required output for any legitimate input in a finite amount of time.

Definition-2: An algorithm is a sequence of steps to solve a problem

This definition can be illustrated by a simple diagram (Figure 1.1).

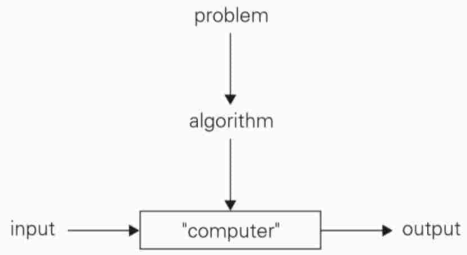


FIGURE 1.1 The notion of the algorithm.

The algorithm is defined as collection of unambiguous instructions occurring in some specific sequence and such an algorithm should produce output for given set of input in finite amount of time.

Properties or Characteristics of Algorithm

An algorithm should have the following properties:

1) **Definiteness:** Each instruction of the algorithm should be clear and unambiguous.

Examples:

- Operations involving division of zero
- Add 5 or 6 to x

2) **Effectiveness:** Every instruction must be very basic or Feasible, so that it can be implemented

3) **Finiteness:** The process should be terminated after a finite number of steps

4) **Input:** There are zero or more quantities that are externally supplied.

5) **Output:** At least one quantity is produced.

Design of an algorithm:

Various steps for algorithm design are:

1. Understanding the problem
2. Decision making on
   a. Capabilities of computational devices
   b. Choice for either exact or appropriate problem solving method
   c. Data structures
   d. Algorithmic strategies
3. Specification of algorithm
4. Algorithmic verification
5. Analysis of algorithm
6. Implementation or coding of algorithm

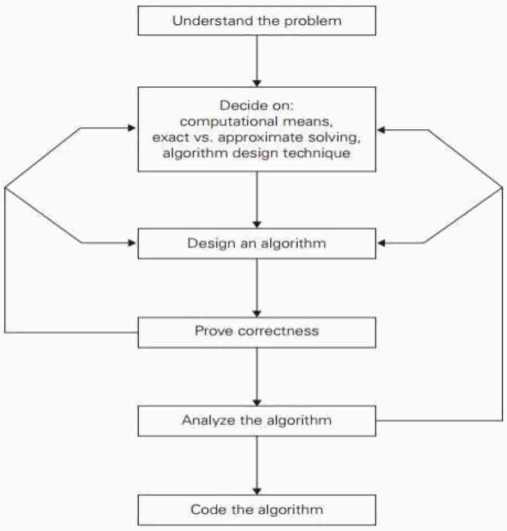


FIGURE 1.2 Algorithm design and analysis process.

1. **Understanding the problem:**
   - In this step first you need to understand the problem statement completely.
   - After understanding we have to find out what are the necessary inputs for solving the problem.
   - The input to the algorithm is called instance of the problem.
   - It is very important to decide the range of inputs so that the boundary values of algorithm get fixed.

- The algorithm should work correctly for all valid inputs.

2. **Decision making:**
   - After finding the required input set for the given problem we have to analyse the input and need to decide certain issues such as:
     a) Capabilities of computational devices:
        - It is necessary to know the computational capabilities of device on which the algorithm is running.
        - Globally we can classify an algorithm from execution point of view such as sequential algorithm and parallel algorithm.
        - For solving complex problems which require huge amount of memory and time it is essential to have proper choice of a computational device which is "space and time efficient".

     b) Choice for either exact or approximate problem solving method:
        - The next important decision is to decide whether to solve the problem exactly or approximately.
        - If the problem needs to be solved correctly then we need exact algorithm.
        - If the problem is so complex that we won't get the exact solution then we need to choose approximation algorithm.

     c) Data structures:
        - Data structures and algorithms work together and these are interdependent.
        - Hence the choice of proper data structure is required before designing the actual algorithm.

     d) Algorithmic strategies:
        - It is a general approach by which many problems can be solved algorithmically.
        - The problems may belong to different areas of computing.
        - Algorithmic strategies are also called as algorithmic techniques or algorithmic paradigm.

     Algorithm design techniques:

     1. Brute force: This is straight forward technique with naive approach.
     2. Divide and conquer: The problem is divided into smaller instances.
     3. Dynamic programming: The results of smaller, reoccurring instances obtained to solve the problem.
     4. Greedy technique: To solve the problem locally optimal decisions are made.
     5. Back tracking: This method is based on the trail and error method. If we want to solve a problem then desired solution is chosen from finite set S.

3. **Specification of algorithm:**

   There are various ways by which we can specify an algorithm. They are:

   a. Natural language
   b. Pseudo code
   c. Flowchart

a. Natural language:

It is very simple to specify an algorithm using natural language.

But many times specification of algorithm by using natural language is not clear .

Example: Write an algorithm to perform addition of 2 numbers.

Step 1: Read the first number say a.

Step 2: Read the second number say b.

Step 3: Add the 2 numbers and store the result in a variable c .

Step 4: Display the result.

b. Pseudo code:

Pseudo code is nothing but a combination of natural language and programming language constructs. A pseudo code is usually more precise than a natural language.

Example: Write an algorithm to perform addition of 2 numbers.

Algorithm sum (a , b)

//problem description : This algorithm performs addition of two integers

//input: two integers a and b.

//output: addition of two integers.

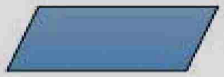
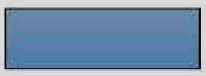
c ← a+b

write(c)

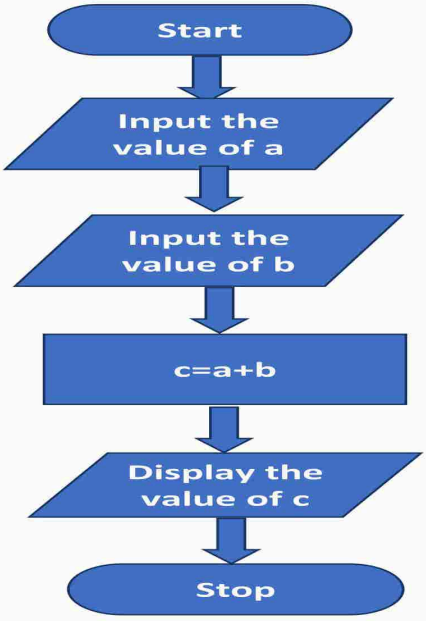
c. Flowchart:

Flowchart is a graphical representation of an algorithm.

Typical symbols used in flowchart are:

Symbol	Name	Function
	Start/end	An oval represents a start or end point
	Arrows	A line is a connector that shows relationships between the representative shapes
	Input/Output	A parallelogram represents input or output
	Process	A rectangle represents a process
	Decision	A diamond indicates a decision

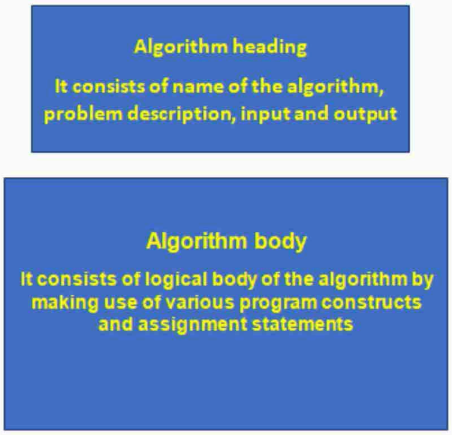
Example: Write an algorithm to perform addition of 2 numbers



4. **Algorithmic verification :**
   - Algorithmic verification means checking correctness of an algorithm.
   - After specifying an algorithm we go for checking its correctness.
   - A common method of proving the correctness of algorithm is by using mathematical induction .
5. **Analysis of algorithm:**
   While analysing an algorithm we should consider following factors:
   a) Time complexity of an algorithm:
      It means the amount of time taken by an algorithm to run. By computing time complexity we come to know whether the algorithm is slow or fast.
   b) Space complexity of an algorithm:
      It means the amount of space (memory) taken by an algorithm .
      By computing space complexity we can analyse whether an algorithm requires more or less space.
6. **Implementation of an algorithm:**
   - The implementation of an algorithm is done by suitable programming language.
   - For example, if an algorithm consists of objects and related methods then it will be better to implement such algorithm using some object oriented programming languages like c ++ or java.
   - While writing a program for given algorithm it is essential to write an optimized code. This will reduce the burden on the compiler.

How to write an algorithm by using pseudocode:

Structure of the algorithm



Example: The following algorithm finds and returns the maximum of n given numbers

```
Algorithm Max(A, n)
// A is an array of size n.
{
Result:=A[1];
for i :=2 to n do
if A[i] >Result then Result := A[i];
return Result;
}
```

Rules for writing algorithms using Pseudocode Conventions:

The conventions of an algorithm are as follows:

1. Comments begin with "//"
2. Blocks are identified with braces: { and }.
3. Identifier begins with a 'letter'.
4. Assignment of values to variables by using:- <Variable> := <Expression>
   (Example a:=10)
5. The Boolean values are 'True' and 'False'.
6. Elements of multidimensional arrays are accessed by using '[' and ']' Ex: A[i,j]
7. Looping statements: while

   while<condition> do

   {

   <statement 1>

   ….<statement n>

   }
8. Looping statements: for

   for variable := value 1 to value 2 do

   {<statement 1>

   ….<statement n>

}

9. Looping statements: repeat

repeat

<statement 1>

….<statement n>

until<condition>

10. Conditional statements: if

if <condition> then <statement>

if <condition> then <statement> else <statement2>

11. Case:

case

{

:<condition 1> : <statement 1>

:<condition 2> : <statement 2>

……:<condition n> : <statement n>

:else: <statement n+1>

}

12. Input and Output are done using the instruction “read” and “write”.

13. An Algorithm consists of heading and body. The syntax is:

Algorithm Name(<parameter List>)

Performance Analysis:

The performance of an algorithm can be measured by:

1. Time Complexity
2. Space Complexity

Time Complexity: How much time it takes to compute

- The time complexity of an algorithm is the amount of completed time to complete its execution.
- The time complexity ‘Tp’ taken by the program ‘P’ is the sum of the ‘Compile Time’ and ‘Run Time’.

Space Complexity: How much memory it takes to compute

- The space complexity of an algorithm is the amount of memory it occupies for completion of execution.

1. Time Complexity:

There are two methods for calculating time complexity:

a. Step Count method

b. Operation Count / Variable Count method

a. Step Count Method

- This method is to count the number of instructions or steps that are used by a given algorithm to perform a particular task.
- The first step is to find 'Steps per Execution (s/e)" and "frequency" of statements.
- For 'executable statements' the 'steps per execution' value is '**1**'.
- For 'non-executable statements' the 'steps per execution' value is '**0**'.

The way of measuring 'step count per execution' are as follows:

1. Statements with no initializations are made with the step count is '**0**' and in case of initialization the step count is '**1**'.
2. 'Comments', 'brackets' and others such as "**begin/end if/end/end for/end while**" all having a step count is '0'.
3. Expressions have a step count of '1'.
4. 'Assignment statement', 'function invocation', 'return statement' and other such as "break/continue/goto" all have step count of '1'.

Example 1:

Step Number	Program	Steps for Execution (s/e)	Frequency	Total Steps (s/e * frequency)
1	Algorithm simple(A,B,C)	0	0	0
2	Begin	0	0	0
3	A:=B+1;	1	1	1
4	C:=A+2;	1	1	1
5	D:=A+B	1	1	1
6	End	0	0	0

Total Time Complexity = 3

Example 2: Sum of elements in array

Step Number	Program	Steps for Execution (s/e)	Frequency	Total Steps (s/e * frequency)
1	Algorithm sum(a,n)	0	0	0
2	{	0	0	0
3	S:=0;	1	1	1
4	for i:=1 to n do	1	N+1	N+1
5	S:=S+a[i];	1	N	N
6	return S;	1	1	1
7	}	0	0	0

Total Time Complexity = 2N+3

Example 3: Matrix Addition

Step Number	Program	Steps for Execution (s/e)	Frequency	Total Steps (s/e * frequency)
1	Algorithm add(a,b,c,m,n)	0	0	0
2	{//a,b,c are arrays	0	0	0
3	for i:=1 to m do	1	m+1	m+1
4	for j:=1 to n do	1	m(n+1)	mn+m
5	c[i,j]:= a[i,j] + b[i,j];	1	mn	mn
6	}	0	0	0

Total Time Complexity = m+1+mn+m+mn = 2mn+2m+1

Example 4: Matrix Transpose

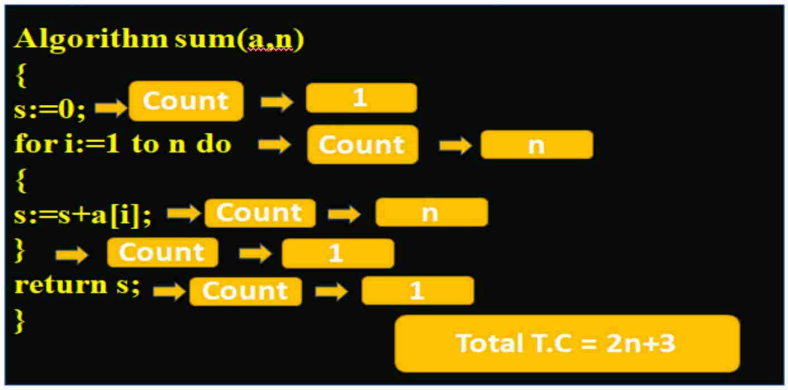
Step Number	Program	Steps for Execution (s/e)	Frequency	Total Steps (s/e * frequency)
1	Algorithm Transpose(a,m,n)	0	0	0
2	{//a is an array	0	0	0
3	for i:=1 to m do	1	m+1	m+1
4	for j:=1 to n do	1	m(n+1)	mn+m
5	t[i,j]:= a[i,j];	1	mn	mn
6	}	0	0	0

Total Time Complexity = m+1+mn+m+mn = 2mn+2m+1

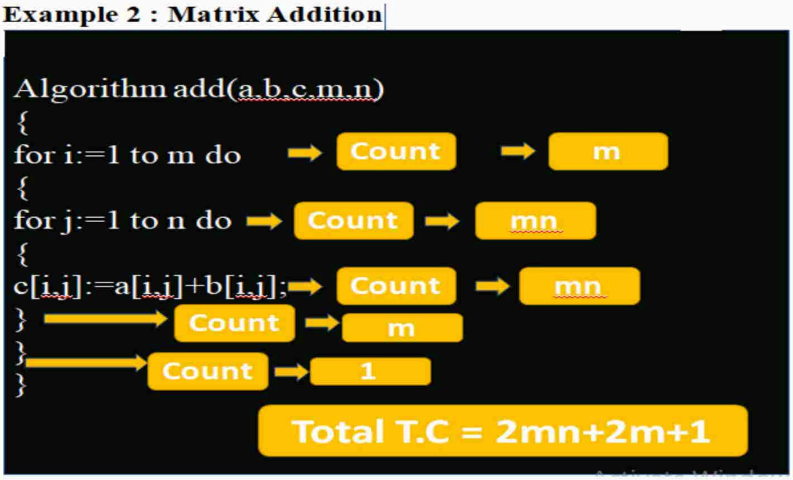
b. Operation Count / Variable Count method

- Instead of counting number of steps, in this method we introduce a “count” variable in the program.
- The “count” variable initially initializes to ‘0’ and it is incremented for each execution of the statement in the program.
- The total value of ‘count’ variable is treated as “time complexity”.

Example 1 : Sum of elements of an array



Example 2 : Matrix Addition



2. Space Complexity

- The space complexity of an algorithm is the amount of memory, it needs to run to completion.
- The space complexity of an algorithm is the sum of two components:
- S(P) = C + Sp
- Space Complexity = Fixed Part (C)+ Variable Part (Sp)

a. Fixed Part:

- It is independent of the characteristics (number , size) of inputs and outputs.
- It varies from problem to problem.
- This part includes the space for the code, for the simple variables, fixed components (∏), for constants, structure variables (array, struct).. etc.

b. Variable Part:

- It is dependent on the problem instances.
- It varies from program-to-program.
- It includes the space for dynamic variables and recursion stack.

Example 1:

```
Algorithm abc(a,b,c)
{
  return a+b*c+(a+b-c)/a+b+4.0
}
```

- This example, is characterized by the specific values of a, b and c.
- Making the assumption one word of memory is adequate to store the values of a,b,c (Fixed Part)
- The space requirement for "variable part" is '0'.
- Therefore the space complexity = C + Sp.
- S(P) = 3 + 0 = 3
- The space complexity = 3

Example 2:

```
Algorithm Sum(a,n)
{
  s:=0
  for i:=1 to n do
  s:=s+a[i];
  return s;
}
```

- One word of memory is required for each n, s and j
- The 'n' space is required for array a[].
- S(P)=C+Sp = 3+n
- Therefore the space complexity of this algorithm is "n+3".

Best Case, Worst Case and Average Case analysis:

92	87	53	10	15	23	67
0	1	2	3	4	5	6

Linear Search Example

1. Best Case: (Algorithm takes minimum possible time)
Searching key element present at first index (Eg. 92)
only one comparison is required
Best Case time, B(n)= O(1) ie. Constant time

2. Worst Case: (Algorithm takes maximum possible time)
Searching key element present at last index (Eg. 67)
'n' comparisons are required
Worst Case time, W(n)= O(n)

3. Average Case: (Algorithm takes average possible time)

All possible case time/ number of cases

Average Case time, A(n)= (1+2+3…n)/n = (n(n+1)/2)/n = (n+1)/2= O(n)

Order of Growth:

Measuring the performance of an algorithm in relation with the input size 'n' is called order of growth.

The primary focus of the order of growth is to find the behaviour of an algorithm.

How do two algorithms differ if their run-time differ by only few seconds or minutes?

Eg. Consider three algorithms A,B and C and respective run-time complexities are $T_A = 7n$, $T_B = 7 \log_{10} n$, $T_C = n^2$ Compare the rates of growth when the input is scaled from n=100 to n=10000

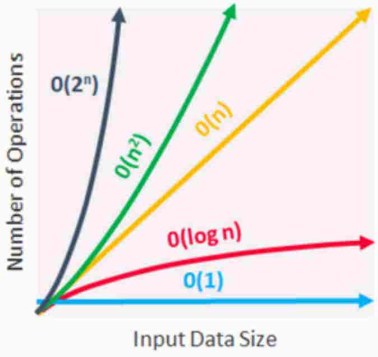
Rate of growth for algorithm A is $T_A(10000)/ T_A(100) = (7x\ 10000)/ (7x\ 100) = 100$

Rate of growth for algorithm B is $T_B(10000)/ T_B(100) = (7x \log_{10} 10000)/ (7x \log_{10} 100) = 2$

Rate of growth for algorithm C is $T_C(10000)/ T_C(100) = (10000^2)/ (100^2) = 10000$

Note : The second Algorithm is the best

	constant	*logarithmic*	*linear*	*N-log-N*	*quadratic*	*cubic*	*exponential*
n	O(1)	O(log *n*)	O(*n*)	O(*n* log *n*)	O(n^2)	O(n^3)	O(2^n)
1	1	1	1	1	1	1	2
2	1	1	2	2	4	8	4
4	1	2	4	8	16	64	16
8	1	3	8	24	64	512	256
16	1	4	16	64	256	4,096	65536
32	1	5	32	160	1,024	32,768	4,294,967,296
64	1	6	64	384	4,069	262,144	1.84×10^{19}



$$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(2^n)$$

Asymptotic Notations (O, Ω, Q)

To choose the best algorithm we need to check the efficiency of each algorithm.

- Efficiency can be measured by computing time complexity of each algorithm.
- Asymptotic notation is a short hand way to represent the time complexity.
- Using asymptotic notations we can give time complexity as fastest possible ,slowest possible or average time .
- Various notations such as O, Ω , θ , o are called as asymptotic notations.

1.Big oh notation (O)
2.Big omega notation (Ω)
3.Big theta notation (θ)
4.Little oh notation (o)

5. Little omega (w)

1. Big oh notation (O):

- It is a method of representing the upper bound of algorithm's running time.
- Using big oh notation, we can give longest amount of time taken by the algorithm to complete.

Definition:

Let f(n) and g(n) be two non-negative functions .

Let n_0 and constant " c " are two integers such that n_0 denotes some value of input and $n \geq n_0$. Similarly c is some constant such that c>0. We can write $f(n) \leq c\ g(n)$ then f(n) = O(g(n)). In other words f(n) is less than g(n) if g(n) is multiple of some constant c.

(OR)

A function f(n) is said to be in O(g(n)) , if f(n) is bounded above by some positive constant multiple of g(n) such that $f(n) \leq c\ g(n)$ for all $n \geq n_0$.

Example-1

3n+2 = O(n)

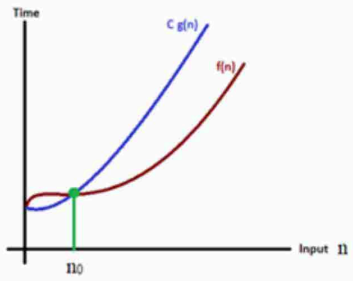
Where f(n) = 3n+2 and g(n) = n

f(n)<=c*g(n)

3n+2 <= c*n

- ❖ if c=3 then 3n+2 <= 3n
- if n=1 then 5<=3 – fail
- If n=2 then 8<=6 – fail
- If n=3 then 11<=9 – fail
- If n=4 then 14<=12 - fail

- ❖ if c=4 then 3n+2 <= 4n
- if n=1 then 5<=4 – fail
- If n=2 then 8<=8 – True
- If n=3 then 11<=12 – True
- If n=4 then 14<=16 – True



❖ **Therefore 3n+2 <= 4n for all n>=2**

Example-2

3n+3 = O(n)

Where f(n) = 3n+3 and g(n) = n

f(n)<=c*g(n)

3n+3 <= c*n

- ❖ if c=3 then 3n+3 <= 3n
- if n=1 then 6<=3 – fail
- If n=2 then 9<=6 – fail
- If n=3 then 12<=9 – fail

- ❖ if c=4 then 3n+3 <= 4n
- if n=1 then 6<=4 – Fail
- If n=2 then 9<=8 – Fail
- If n=3 then 12<=12 – True
- If n=4 then 15<=16 – True

❖ **Therefore 3n+2 <= 4n for all n>=3**

2. Big omega notation (Ω):

- It is used to represent the lower bound of algorithms running time.
- Using big omega notation, we can denote shortest amount of time taken by the algorithm.

Definition:

A function f(n) is said to be in Ω(g(n)) , if f(n) is bounded below by some positive constant multiple of g(n) such that f(n) ≥ c g(n) for all $n \geq n_0$, c >0 , $n_0 \geq 1$. It is denoted by f(n) = Ω(g(n)).

Example:

3n+2 = Ω(n)

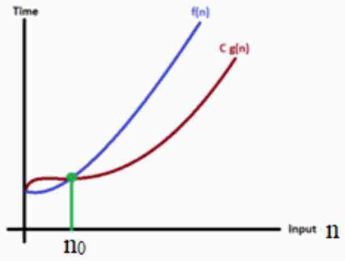
Where f(n) = 3n+2 and g(n) = n

f(n)>=c*g(n)

3n+2 >= c*n

- ❖ if c=3 then 3n+2 >= 3n
- if n=1 then 5>=3 – True
- if n=2 then 8>=6 – True
- if n=3 then 11>=9 – True

❖ **Therefore 3n+2 >= 3n for all n>=1**



3. Big theta notation (θ):

- The theta notation is denoted by "θ".
- The function f(n) should vary in between upper bound and lower bound.

Definition:

Let f(n) and g(n) be two non-negative functions , there are two positive constants c_1 and c_2 such that c_1 g(n) ≤ f(n) ≤ c_2 g(n) then we say that f(n) = θ(g(n)).

Example:

3n+2 = Θ(n)

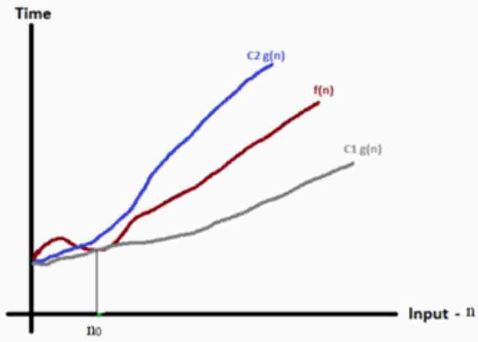
Where f(n) = 3n+2 and g(n) = n

c_1 n <= 3n+2 <= c_2 n

- ❑ 3n+2 >= 3n for all n>=2
- ❑ 3n+2<=4n for all, n>=2
- ❑ **Note:** 'n' should be equal in both cases

❖ **C_1=3, C_2=4 and n=2**

❖ **Therefore 3n<= 3n+2<=4n for all n>=2**



4. Little oh notation (o):

- It is used very rarely, hence the value of c is very small.
- For example, c=0.001, the little oh notation can be used instead of big oh notation as the little oh notation represents loose bound.

Definition:

The function f(n) = o(g(n)) if and only if

$$\lim_{n\to\infty} \frac{f(n)}{g(n)} = 0$$

Example: f(n) = 3n+2 and g(n)=n²

Therefore 3n+2 = o(n²)

$$\lim_{n\to\infty} \frac{3n+2}{n^2} = \lim_{n\to\infty} \frac{n(3+\frac{2}{n})}{n^2} = \lim_{n\to\infty} \frac{(3+\frac{2}{n})}{n} = \lim_{n\to\infty} \frac{1}{n}(3+\frac{2}{n}) = 0(3+2/n) = 0$$

- **5. Little 'omega' (w):**

The function f(n) = w(g(n)) if and only if

$$\lim_{n\to\infty} \frac{g(n)}{f(n)} = 0$$

Example: f(n) = n³ and g(n)=6n²+18

Therefore $n^3 = w(6n^2 + 18)$

$$\lim_{n\to\infty} \frac{6n^2+18}{n^3} = \lim_{n\to\infty} \frac{n^2(6+18/n^2)}{n^3} = \lim_{n\to\infty} \frac{1}{n}(6+\frac{18}{n^2}) = 0(6+\frac{18}{n^2})=0$$

Amortized Analysis

- An amortized analysis is finding the average running time per operation over a worst case sequence of operations .
- Types of amortized analysis are:
  1. Aggregate analysis
  2. Accounting method
  3. Potential method

- Suppose a sequence I1, I2, D1, I3, I4, I5, I6, D2, I7 of insert and delete operations are performed on sequence.
- Assume that 'actual cost' of insertion operation is '1' and for deletion operation D1, D2 are 8 and 10 respectively.
- So the total cost of the above sequence is 25.
- Now, by using amortized analysis we can reduce the charged cost of some operations and increased that of other.
- Amortized analysis gives the average performance of each operation in worst case.
- Let assume the 'amortized cost' is greater than 'actual cost'.

  amortized(i) >= actual(i)

  amortized(i)-actual(i)>=0

- In the above equation, amortized(i) and actual(i) denotes amortized and actual complexities of ith operation.
- The sum of amortized complexities of all operations in any sequence of operations be grater than or equal to their sum of the actual complexities.

$$\sum_{1 \le i \le n} amortized(i) \ge \sum_{1 \le i \le n} actual(i)$$

- Relative to actual and amortized cost of each operation in any sequence we define a potential function p(i) is as follows:

p(i) = amortized(i) – actual(i) + p(i-1)

- In the above equation 2, the ith operation causes the potential function to change by the differences between amortized cost and actual cost for all 1<=i<=n

$$\sum_{1 \le i \le n} P(i) = \sum_{1 \le i \le n} (amortized(i) - actual(i) + P(i-1)) = \sum_{1 \le i \le n} (P(i) - P(i-1)) = \sum_{1 \le i \le n} (amortized(i) - actual(i))$$

Expand $\sum_{1 \le i \le n} (p(i) - p(i-1))$

= p(1)+p(2)+p(3)+…..p(n) – p(0) – p(1) – p(2)…… – p(n-1)

= p(n) – p(0)

Therefore $p(n) - p(0) = \sum_{1 \le i \le n} (amortized(i) - actual(i))$

- So, under the assumption of p(0)=0, the potential p(i) is the amount by which ith operation has been overcharged.
- P(n) – p(0) >=0

Example: Maintenance Contract

In January, you buy a new car from a dealer who offers you the following maintenance contract:

1. **$50** each month for January, February, April, May, July, August, October, November.
2. **$100** for every March, June and September
3. **$200** for every December

Now our goal is to obtain an upper bound on the cost of this maintenance contract as a function of number of months

We can solve amortized analysis by using 3 methods:

1. Aggregate Method
2. Accounting Method
3. Potential Method

1. **Aggregate Method**

The amortized cost of each operation is equal to

=(upper bound on sum of actual cost) / n

So by using aggregate method:

$$= 200(\frac{1*n}{12}) + 100(\frac{3*n}{12}) + 50(\frac{8*n}{12})$$

$$= (\frac{200n+300n+400n}{12}) = (\frac{900n}{12}) = 75n$$

- **$50** each month for January, February, April, May, July, August, October, November.
- **$100** for every March, June and September
- **$200** for every December

- Therefore the amortized cost for every month is **75**.

Month	Jan	Feb	Mar	Apr	May	June	July	Aug	Sep	Oct	Nov	Dec
Actual Cost	50	50	100	50	50	100	50	50	100	50	50	200
Amortized Cost	75	75	75	75	75	75	75	75	75	75	75	75
Potential Function P(i)	25	50	25	50	75	50	75	100	75	100	125	0

The potential cost for every month is:

- P(Jan) = 75-50+0 = 25
- P(Feb) = 75-50+25=50
- P(June)=75-100+75= -25+75 = 50
- P(Sep) = 75-100+100 = 75
- P(Dec) = 75-200+125 = -125+125 = 0

p(i) = amortized(i) – actual(i) + p(i-1)

2. **Accounting method**

- In this method, let we try to take amortized cost as $70. To check whether we can get potential cost >=0 or not.
- If we get potential value as ‘negative’ then amortized cost $70 is not correct for analysis.

Month	Jan	Feb	Mar	Apr	May	Jun	Jul	Aug	Sep	Oct	Nov	Dec
Actual Cost	50	50	100	50	50	100	50	50	100	50	50	200
Amortized Cost	70	70	70	70	70	70	70	70	70	70	70	70
Potential Function P(i)	20	40	10	30	50	20	40	60	30	50	70	- 60

The potential function: p(i) =

p(i) = amortized(i) – actual(i) + p(i-1)

- P(Jan) = 70-50+0=20
- P(Feb) = 70-50+20=40
- P(Dec) = 70-200+70 = **- 60 (Negative Value)**

3. **Potential method**

- In this method, we define a potential function for the analysis which represents the cumulative difference between actual and amortized cost.
- The potential function based on the table of 'aggregate method' as follows:

p(i) = amortized(i) – actual(i) + p(i-1)

- amortized cost = actual(i) + p(i) - p(i-1)

- Amortizedcost(jan) = 50+25-0 = 75
- Amortizedcost(feb) = 50+50-25 = 75
- Amortizedcost(March) = 100+25-50 = 75
- Amortizedcost(April) = 50+50-25 = 75
- AmortizedCost(Dec) = 200+0-125 = 75

Month	Jan	Feb	Mar	Apr	May	June	July	Aug	Sep	Oct	Nov	Dec
Actual Cost	50	50	100	50	50	100	50	50	100	50	50	200
Amortized Cost	75	75	75	75	75	75	75	75	75	75	75	75
Potential Function P(i)	25	50	25	50	75	50	75	100	75	100	125	0